AI Assignment 1

COMPREHENSIVE PATHFINDING ALGORITHM REPORT:

Comparative Analysis of Uninformed and Informed Search Strategies on Grid-Based Maps

1. Architectural Design & Implementation Logic

The primary goal of this implementation is to construct a scalable and dynamic grid-based pathfinding environment to observe the behaviors of four graph-search strategies. The environment models space as a structured grid where coordinates map directly to nodes in an implicit graph framework.

Graph Modeling & Coordinate System

The application maps a discrete grid system represented internally as a matrix of rows and columns. Each element is defined by a coordinate tuple (r, c). Obstacles are tracked using a Python set structure containing coordinates designated as closed. This allows for an O(1) membership validation routine when calculating eligible neighboring configurations during graph exploration loops. Edge weights between any two adjacent open cells are constant and uniform, evaluated at 1.0.

Data Structures and Queue Frameworks

- Breadth-First Search (BFS): Evaluates nodes strictly by utilizing a standard First-In-First-Out (FIFO) queue array. The search maps out layers level by level, ensuring that nodes closer to the initial state are exhausted before deeper nodes are queued.
- Uniform-Cost Search (UCS): Operates using a priority queue backed by Python's native heapq module. Cells are ordered dynamically by their accumulated path cost g(n) from the root node. This guarantees an optimal extraction sequence.
- Greedy Best-First Search (GBFS): Utilizes a priority queue ordered exclusively by an absolute heuristic assessment f(n) = h(n). It avoids historical path evaluation, preferring localized immediate structural evaluation.
- A* Search: Leverages a priority queue tracking the unified formula f(n) = g(n) + h(n). This methodology successfully bounds raw heuristic projections with precise historical navigation tracking.

2. Algorithmic Matrix & Behavioral Performance

A core aspect of this study involves mapping out how structural metrics change based on algorithmic strategy. Below is the systematic comparison table of the parameters under identical environment configurations:

Algorithm	Evaluation f(n)	Optimality	Frontier Shape	Performance Character
BFS	None (FIFO Layer)	Optimal	Radial Circular Wave	Exhaustive spatial expansion across empty spaces.
UCS	f(n) = g(n)	Optimal	Uniform Expanding Ring	Behaves identically to BFS given standard uniform weights.
GBFS	f(n) = h(n)	Non-Optimal	Linear Direct Beam	Highly focused vector lines; easily trapped by complex layouts.
A*	f(n) = g(n) + h(n)	Optimal	Focused Elliptical Shape	Balances path depth with proximity to yield high efficiency.

3. Heuristic Assessment

Informed variants rely completely on heuristic projections to narrow structural verification spaces. The environment evaluates two distinct geometric options:

Manhattan Distance: Designed specifically for orthogonally bound systems where diagonal vectors are prohibited. It calculates grid distance via absolute geometric separation: h(n) = |r1 - r2| + |c1 - c2|. This heuristic matches the actual topology of the grid, ensuring an admissible and consistent estimate that minimizes unnecessary node exploration.

Euclidean Distance: Measures the straight-line distance across space: h(n) = sqrt((r1 - r2)^2 + (c1 - c2)^2). While geometrically accurate, it can occasionally under-represent actual step sequences on strict 4-directional grids, leading to a slightly broader search frontier compared to the Manhattan heuristic.

4. Analytical Evaluation of Scenarios

The behavioral variances between algorithms become highly apparent when comparing best-case and worst-case grid maps:

Unobstructed Grid Layouts (Best-Case)

When running on a completely open map, GBFS and A* show exceptional efficiency. GBFS targets the goal directly along a clean diagonal line, expanding only the minimal number of nodes along its path. A* mirrors this direct route, using its history cost to prevent any stray path exploration. In stark contrast, BFS and UCS expand indiscriminately across the open space, filling a large circular area before hitting the target node.

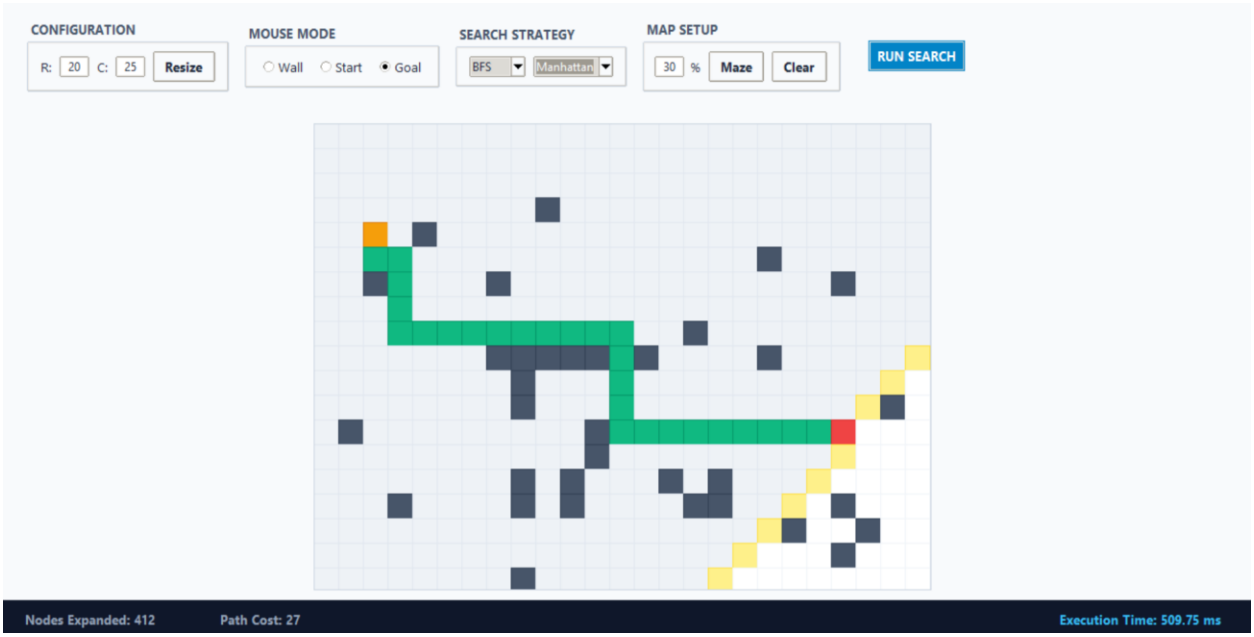
Concave Barrier Formations (Worst-Case)

When a large, U-shaped obstacle blocks the direct path, the differences change dramatically. GBFS exhibits severe path distortion; because it evaluates options based solely on straight-line distance, it drives deep into the center of the trap, expanding every enclosed node before backtracking out. A* handles the trap much more effectively. While its heuristic draws it toward the barrier, its integrated step-cost tracker g(n) flags the rising costs, forcing it to route cleanly around the obstacle. BFS and UCS exhaustively fill the entire open canvas until the boundary edge is circumvented, maintaining path optimality at the expense of computational performance.

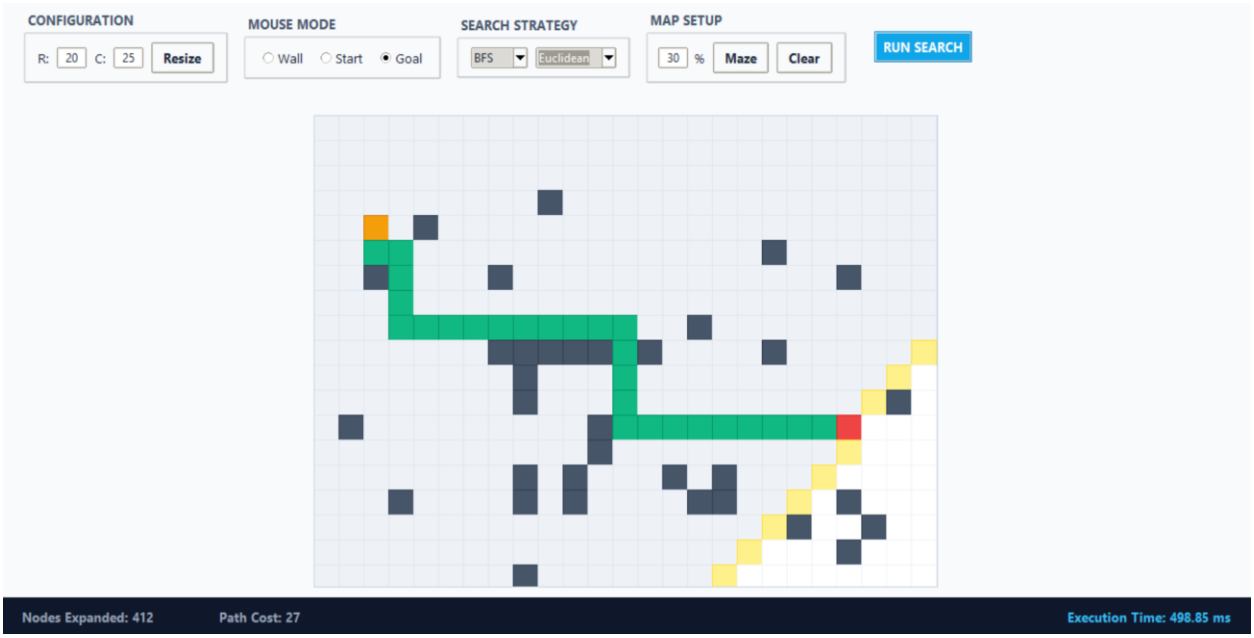
Screenshots:

Following are the screenshots with respective to each Algorithm and working scenario.

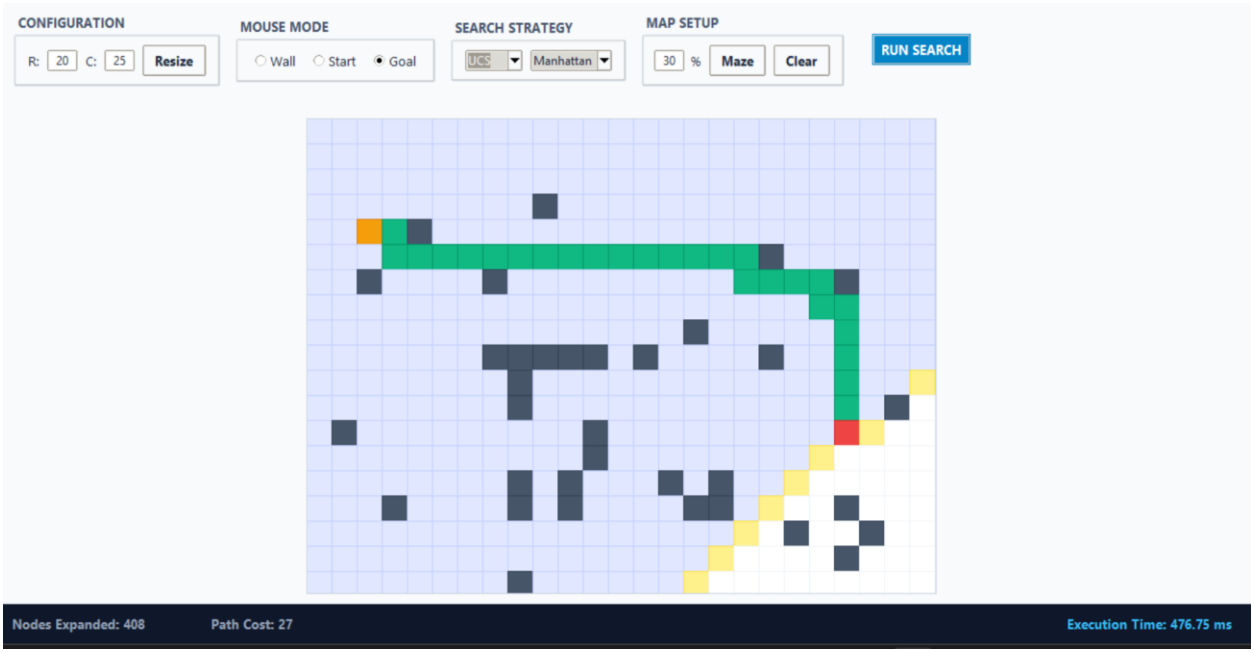
- **BFS Manhattan**



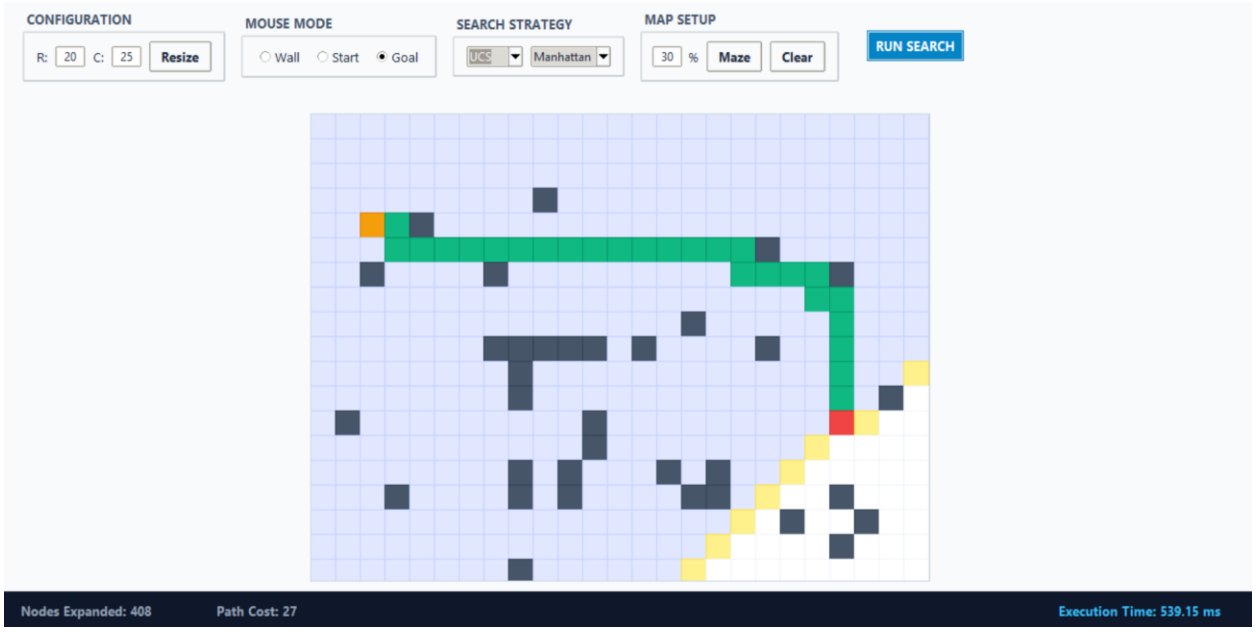
- **BFS Euclidean**



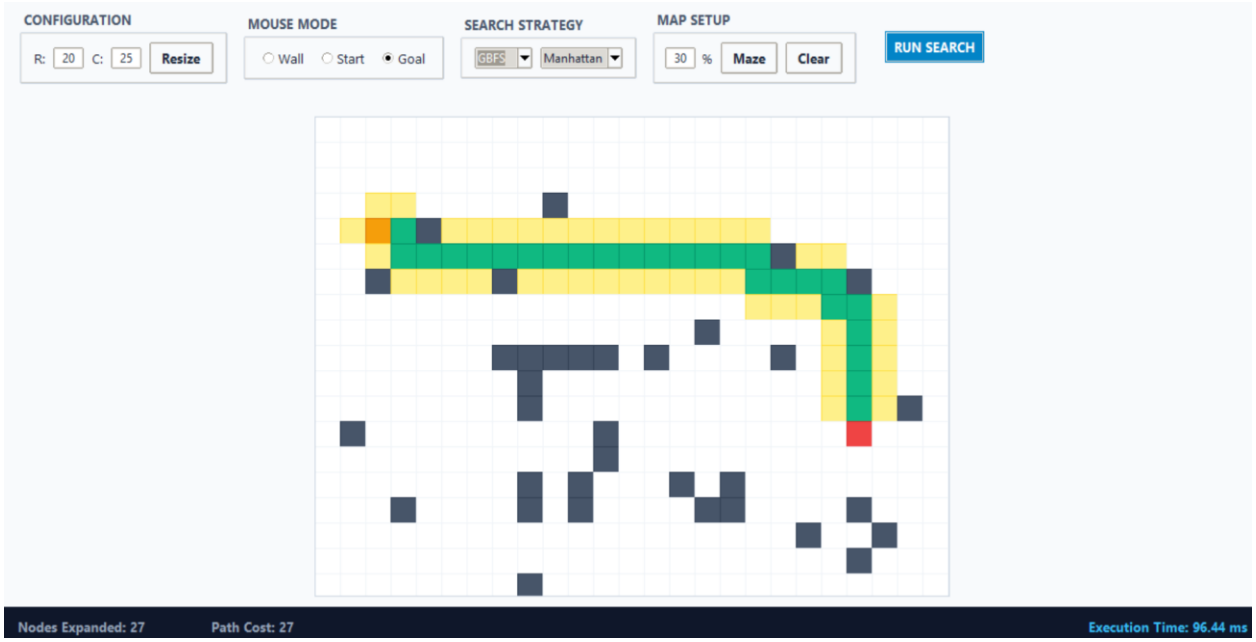
- **UCS Manhattan**



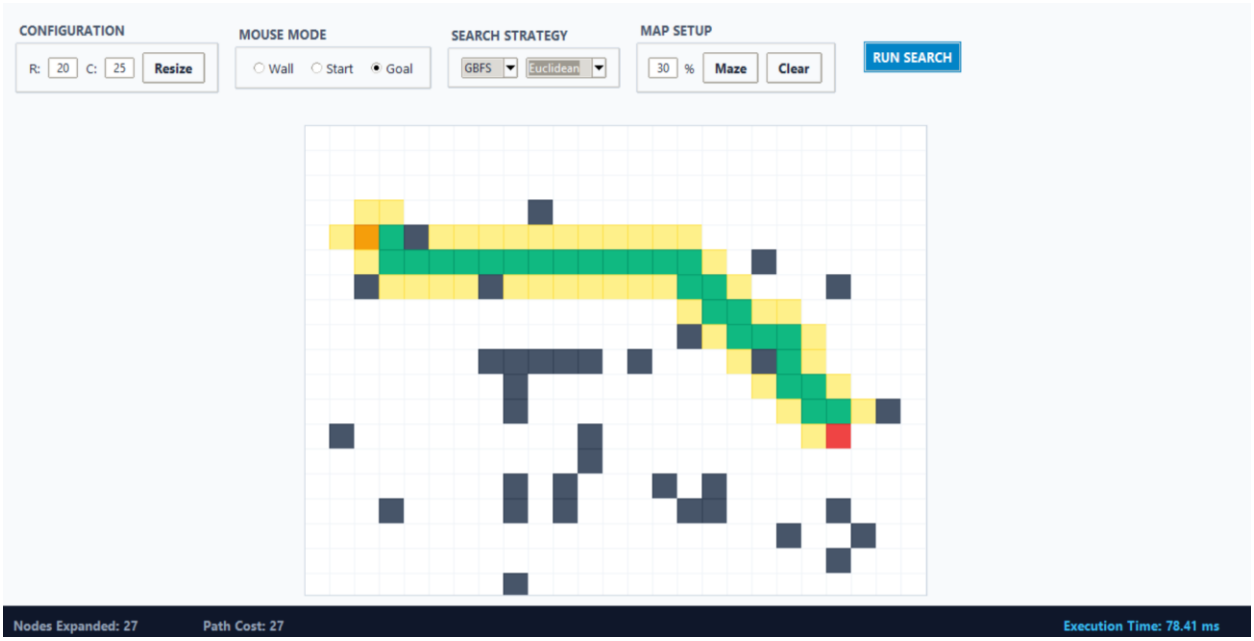
- **UCS Euclidean**



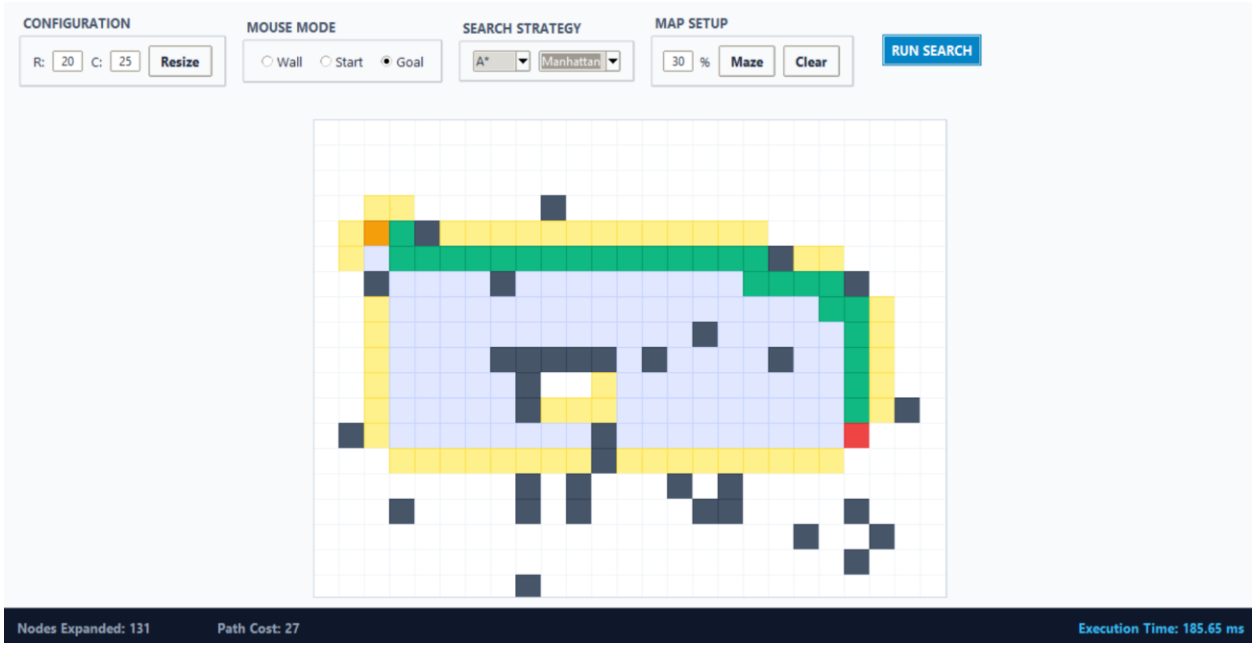
- **GBFS Manhattan**



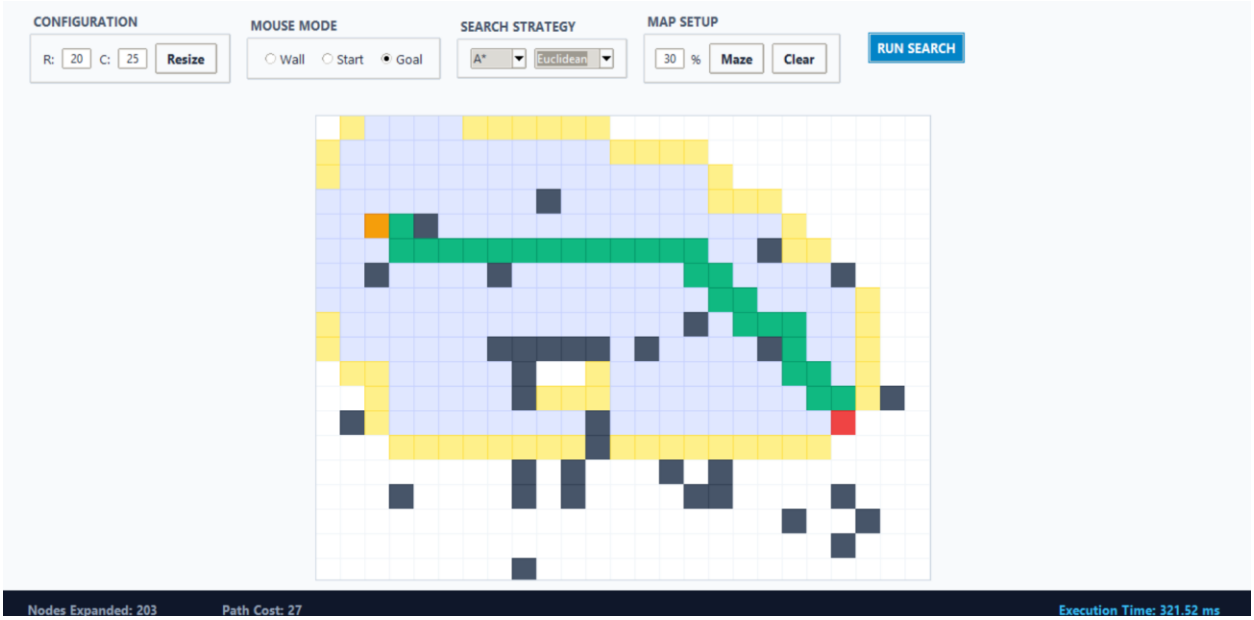
- **GBFS Euclidean**



- **A* Manhattan**



- **A* Euclidean**


CONFIGURATION
R: 20 C: 25 Resize
MOUSE MODE
Wall Start Goal
SEARCH STRATEGY
A* Euclidean
MAP SETUP
30 % Maze Clear
RUN SEARCH
Nodes Expanded: 203
Path Cost: 27
Execution Time: 321.52 ms